

Assessing Quantum Layout Synthesis Tools via Known Optimal-SWAP Cost Benchmarks

Shuohao Ping¹, Wan-Hsuan Lin¹, Daniel Bochen Tan², Jason Cong¹

¹Computer Science Department, University of California, Los Angeles, CA, USA,

²Harvard University Cambridge, MA, USA

{sp1831, wanhsuanlin}@ucla.edu; danieltan@g.harvard.edu; cong@cs.ucla.edu

Abstract—Quantum layout synthesis (QLS) is a critical step in quantum program compilation for superconducting quantum computers, involving the insertion of SWAP gates to satisfy hardware connectivity constraints. While previous works have introduced SWAP-free benchmarks with known-optimal depths for evaluating QLS tools, these benchmarks overlook SWAP count—a key performance metric. Real-world applications often require SWAP gates, making SWAP-free benchmarks insufficient for fully assessing QLS tool performance. To address this limitation, we introduce QUBIKOS, a benchmark set with provable-optimal SWAP counts and non-trivial circuit structures. For the first time, we are able to quantify the optimality gaps of SWAP gate usages of the leading QLS algorithms, which are surprisingly large: LightSABRE from IBM delivers the best performance with an optimality gap of 63x, followed by ML-QLS with an optimality gap of 117x. Similarly, QMAP and t|ket exhibit significantly larger gaps of 250x and 330x, respectively. This highlights the need for further advancements in QLS methodologies. Beyond evaluation, QUBIKOS offers valuable insights for guiding the development of future QLS tools, as demonstrated through an analysis of a suboptimal case in LightSABRE. This underscores QUBIKOS’s utility as both an evaluation framework and a tool for advancing QLS research.

I. INTRODUCTION

Superconducting qubit has emerged as a promising solution for realizing large scale quantum computing. However, the limited qubit connectivity in superconducting quantum computers restricts two-qubit gates to specific pairs of qubits. In order to transform quantum circuits into hardware-executable forms, we need quantum layout synthesis (QLS) to insert SWAP gates that enable arbitrary two-qubit gates in the circuit. However, these SWAP gates increase circuit size and depth, reducing overall fidelity. Thus, minimizing the number of inserted SWAP gates is a primary objective of QLS.

The QLS problem is proved to be NP-hard [1]. The existing methods fall into two classes: exact approaches or heuristic algorithms. The exact approaches formulate a QLS problem into a constraint satisfaction problem and can obtain optimal results [2]–[10] but it cannot scale up to solve problems with a hundred qubits. To address scalability issues, several heuristic algorithms are proposed [1], [11]–[27]. These methods are efficient, yet exhibit performance degradation as the problem size increases. While there is a clear runtime advantage of heuristic algorithms over exact methods, the trade-off in solution quality remains uncertain due to the lack of benchmarks with known optimal SWAP counts. Measuring the optimality gap using exact tools is an alternative, but this approach becomes impractical for large circuits due to scalability issues.

Therefore, having a benchmark set with known optimal SWAP counts is crucial for enabling solid comparison of QLS tools.

Few studies have attempted to establish such benchmark sets. Tan and Cong introduced the QUEKO benchmark with known-optimal depth and zero SWAP gates [28]. However, this benchmark is limited in scope; it can be addressed by QLS tools employing subgraph isomorphism and fails to evaluate tool performance on circuits that require SWAP gates. Li et al. proposed the QUEKNO benchmark [29], which generates circuits requiring SWAP gates by deriving a series of mappings from an initial mapping and selected SWAP operations. However, these circuits do not have known optimal SWAP counts, and the solutions are only claimed to be near-optimal without rigorously quantifying the optimality gap. As a result, QUEKNO cannot measure the optimality gap of QLS tools. To overcome the limitations of existing benchmarks, we propose QUBIKOS (QUAntum Benchmark wIth KNown Optimal Swap counts), which is the first benchmark set with a known optimal non-zero SWAP count. It permits a more comprehensive assesment of QLS tools and supports the development of future advancements in this area.

With QUBIKOS, we evaluate four QLS tools on four architectures and observe the optimality gaps. We find that as the size of the architecture grows, the optimality gap also grows from 1x to 233.97x even for the best performing tools. This shows an urgent need to have a more effective QLS tool to fully utilize the power of large systems. Furthermore, for architectures with a similar size, the structure of the architecture also affects the optimality gap. For instance, the optimality gap for IBM Rochester is seven times larger than Google Sycamore as a result of sparse connectivity and fewer axes of symmetry.

QUBIKOS provides a qualitative evaluation on the performance of QLS tools by measuring the optimality gap between QLS results and optimal results. We can control the circuit size and number of optimal SWAP gates in the QUBIKOS circuit, which allows us to test the QLS tools under different conditions. QUBIKOS also offers insights on improving the performance of QLS tools by analyzing examples when QLS tools fails to find optimal SWAP gate counts.

The rest of this paper is organized as follows. Section II introduces the quantum layout synthesis problem. Section III details the construction of QUBIKOS benchmark. Section IV evaluates existing QLS tools with QUBIKOS, and Section V concludes the paper.

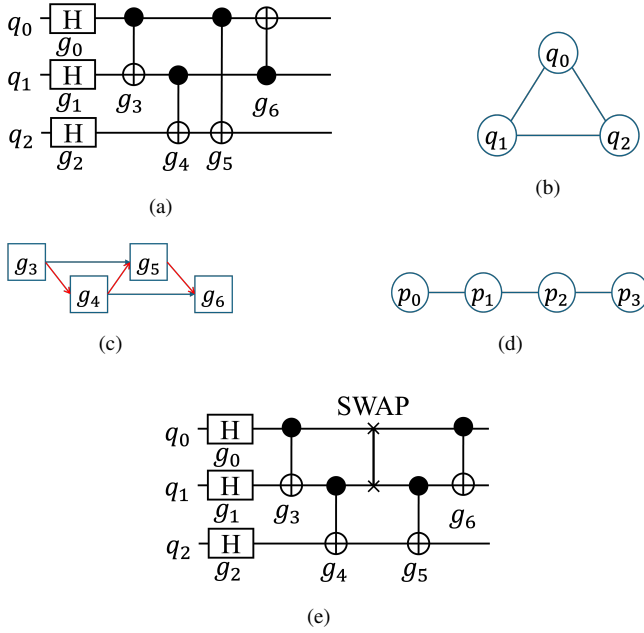


Figure 1. (a) An example of a quantum circuit. (b) The interaction graph for the circuit shown in (a). (c) Gate dependency graph derived from the circuit. The red edges mark a dependence chain from g_3 to g_5 . (d) A device coupling graph with 4 physical qubits in a line architecture. (e) A transpiled circuit of (a) with one inserted SWAP gate for the line architecture with mapping $q_0 \rightarrow p_0, q_1 \rightarrow p_1$, and $q_2 \rightarrow p_2$.

II. QUANTUM LAYOUT SYNTHESIS

The QLS problem arises from the need to map gates in a quantum circuit to a superconducting quantum processor characterized by a coupling graph and perform necessary transformations to the circuit via inserting SWAP gates. The inputs to the QLS problem are: *quantum circuit* and *coupling graph*.

A *quantum circuit* is defined as a sequence of single and two-qubit gates. In this paper, we denote the set of program qubits by Q , a program qubit by $q \in Q$, the set of single-qubit gates by $\mathcal{G}_1$, the set of two-qubit gates by $\mathcal{G}_2$, and the overall gate set by $\mathcal{G} = \mathcal{G}_1 \cup \mathcal{G}_2$. In addition, for a two-qubit gate $g(q, q') \in \mathcal{G}_2$, we use $g[0] = q$ and $g[1] = q'$ to denote the two program qubits g operates on. Figure 1(a) is an example of a quantum circuit where H gates are single qubit gates, and CNOT gates are two-qubit gates.

A *coupling graph* $G_C(P, E_P)$ describes the connectivity of physical qubits for a given hardware where P is a set of physical qubits and E_P is a set of edges, where two-qubit gates can be performed. Figure 1(d) depicts a coupling graph. We define the function $Neighbor(v, G)$ to be the set of adjacent vertices of v in G . For example, the adjacent physical qubits of a physical qubit $p \in P$ is denoted as $Neighbor(p, G_C)$.

According to a quantum circuit C , we define the *interaction graph* $G_I(Q, E_Q)$, where each program qubit is a node and an edge exists between q and q' if $g(q, q') \in \mathcal{G}_2$. Figure 1(b) demonstrates the interaction graph of the circuit in Figure 1(a). The neighbor of a program qubit $q \in Q$ in the interaction graph

$G_I(Q, E_Q)$ is designated by $Neighbor(q, G_I)$.

The *gate dependency graph* $D(\mathcal{G}_2, E_G)$ is a directed acyclic graph that represents the execution order of two-qubit gates. Single-qubit gates are excluded from D since they do not impose connectivity constraints, and can be inserted back to the circuit after QLS. Nodes in D represent two-qubit gates in the circuit and an edge (g, g') is drawn if g' can be executed immediately after g . Figure 1(c) shows an example of D for the circuit in Figure 1(a). There is an edge from g_3 to g_4 because they both act on q_1 , and g_4 comes after g_3 . In general, for any given gates $g, g' \in \mathcal{G}_2$, if g' must execute before g , there exists a path from g' to g in D . We define the set of previous gates of gate g as $Prev(g) = \{g' | \exists \text{ path from } g' \text{ to } g \text{ in } D\}$.

The output of the QLS problem is a *transpiled circuit* with inserted SWAP gates, and an initial mapping from program qubits to physical qubits $f : Q \rightarrow P$. Figure 1(e) illustrates a valid QLS result using one SWAP to map the circuit in Figure 1(a) to the coupling graph in Figure 1(d). A transpiled circuit can be written in the form of $C_0 \cdot T_0 \cdot C_1 \cdot T_1 \dots T_{n-1} \cdot C_n$, where C_i is a partial gate sequence with $C_0 \cdot C_1 \dots C_n = C$, and T_i is a SWAP gate that implements mapping transformation.

III. QUBIKOS CIRCUIT CONSTRUCTION

To construct a circuit with an optimal SWAP count of n given a coupling graph G_C , the first step is to generate n circuit segments. Each requires exactly one SWAP gate under all possible mappings, which means the interaction graph is not isomorphic to any subgraph of G_C , as such isomorphisms would allow execution without SWAP insertion. Figure 2(c) is a non-isomorphic interaction graph to the coupling graph G_C displayed in Figure 2(a) as the interaction graph contains a node with degree 5, while the maximum degree in G_C is 4.

However, simply concatenating these graphs does not guarantee an optimal SWAP count. Thus, the second step is to construct gate dependencies both within and between the interaction graphs and enforce that gates from different interaction graphs execute in serial. This guarantees that when we concatenate the segments to construct the final circuit, the corresponding optimal SWAP count equals the sum of the optimal SWAP counts for each segment.

Figure 3 shows a gate dependency graph of a circuit that requires 2 SWAP gates. Here, g_4 and g_{15} partition the circuit into two sets: $g_0 - g_4$ and $g_5 - g_{15}$, with no gates from different sets executed in parallel. These sets form two non-isomorphic interaction graphs that must execute serially, ensuring each requires one SWAP gate to resolve connectivity, and no single SWAP can resolve conflicts in both graphs.

Repeating this process n times, we can generate a circuit C with n SWAP gates. Finally, we can insert additional gates that do not increase the optimal SWAP count, allowing for arbitrary circuit structures and making it more challenging to solve. The following sections provide detailed examples of the subroutines.

A. Non-isomorphic Interaction Graph Generation

The first step constructs non-isomorphic interaction graphs relative to a given architecture G_C . Starting with a random

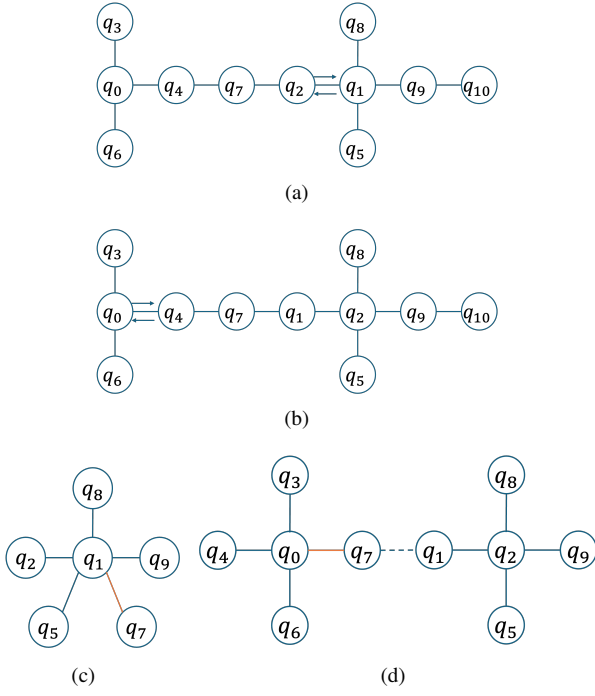


Figure 2. (a) The device coupling graph G_C with a random initial mapping. Arrows represent the first SWAP operation. (b) The device coupling graph with qubit mapping after $\text{SWAP}(q_1, q_2)$ on the initial mapping. Arrows indicate the second SWAP operation. (c) The interaction graph $G_{I_1}(Q, E_{Q_1})$ that enforces $\text{SWAP}(q_0, q_4)$. The orange edge is the special gate. (d) The interaction graph $G_{I_2}'(Q, E'_{Q_2})$ that only considers gates after the first SWAP. The orange edge is the special gate in E_{Q_2} . The dotted line is the inserted gate. Without it, the coupling graph is G_{I_2} .

initial mapping, we select an adjacent qubit pairs for SWAP gate. The SWAP is designed to enable at least one qubit in the pair to interact with a new neighbor, ensuring the SWAP is essential. If no new interactions are introduced, the SWAP is redundant. Such edges always exist unless the coupling graph is fully connected. Figure 2(a) illustrates an example of the initial mapping on the coupling graph with SWAP gates on (q_1, q_2) , enabling new interactions. For selecting subsequent SWAP gates, we need to transform the mapping based on the selected SWAP and choose an adjacent qubit pair in the new mapping that satisfies the condition discussed above. Figure 2 shows the new mapping after $\text{SWAP}(q_1, q_2)$, and we choose edge (q_0, q_4) for the next SWAP.

To enforce a SWAP between q_1 and q_2 , we construct an interaction graph based on q_1 . First, q_1 must interact with all its current neighbors, ensuring it has no free edges to form new interactions. This requires a two-qubit gate on $g(q_1, q_2)$, $g(q_1, q_5)$, $g(q_1, q_8)$, and $g(q_1, q_9)$. Next, we identify a qubit that is not a neighbor of q_1 before the SWAP gate but becomes one afterward, e.g., q_7 , and add the *special gate* $g(q_1, q_7)$ in the circuit. The resulting interaction graph is displayed in Figure 2(c), and is not isomorphic to any subgraph of G_C . Even though the circuit before the special gate does not need a SWAP gate, it forces the circuit to require one prior to the its execution. Any gate in the interaction graph could serve

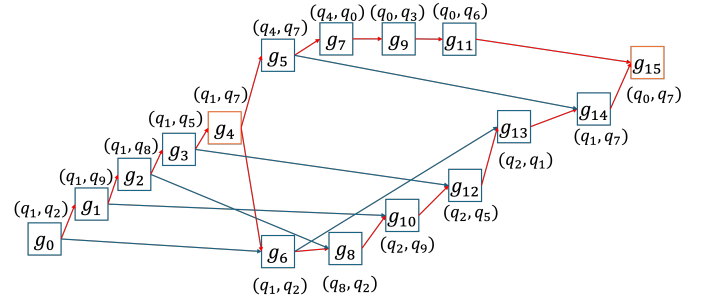


Figure 3. Gate dependency graph D for circuit C . The orange boxes represent the special gates. The red edges show the required dependence relation for g_4 and g_{15} .

as the special gate, provided the remaining four gates form an isomorphic subgraph. The special gate must execute last to ensure all other gates in the interaction graph are completed beforehand. Otherwise, it may alter the interaction graph by removing an edge and eliminate the need for a SWAP gate. Section III-B will present how to impose the dependency constraint.

Alternatively, we could use q_2 to build the interaction graph. However, simply having q_2 interact with all its neighbors plus one new neighbor qubit is insufficient. In this case, q_2 would have a degree of 3 in the interaction graph, and a mapping could exist that avoids SWAP gates by placing q_2 on a degree 3 or degree 4 node in G_C . To prevent this, all program qubits mapped to physical qubits with degrees higher than 2 must also interact with all their neighbors. This ensures all higher-degree physical nodes are occupied, making it impossible to find a new mapping that avoids SWAP gates by placing q_2 on a higher-degree physical qubit.

Next, we want to construct a circuit C to compel $\text{SWAP}(q_0, q_4)$. After $\text{SWAP}(q_1, q_2)$, the mapping is updated as in Figure 2(b). Using q_0 as the starting point for constructing the non-isomorphic graph, we ensure q_0 interacts with all its neighbors, generating gates $g(q_0, q_3)$, $g(q_0, q_6)$, and $g(q_0, q_4)$. Then, $g(q_0, q_7)$ serves as the special gate that enforce $\text{SWAP}(q_0, q_4)$. In addition, q_2 must interact with all its neighbors since q_2 is mapped to a physical qubit whose degree is larger than the one that q_0 is mapped to. Figure 2(d) shows the interaction graph of G_{I_2} . Note that the dotted line is current not part of the interaction graph, but will be added in the next subroutine.

B. Gate Dependency Relation

Given the interaction graphs G_{I_1}, G_{I_2} generated by the first step, we build the QUBIKOS circuits by enforcing the gate dependency within and between interaction graphs. Considering the gates in G_{I_1} , since all these gates act on q_0 , we can reorder them so that the special gate depends on all other gates, ensuring that the special gate forces a SWAP operation. For example, in the gate sequence $C_1 = [g_0(q_1, q_2), g_1(q_1, q_9), g_2(q_1, q_8), g_3(q_1, q_5), g_4(q_1, q_7)]$, the special gate $g_4(q_1, q_7)$ has to execute after all other gates.

Reordering alone does not always ensure the desired gate dependencies. For instance, in G_{I_2} , the special gate (q_0, q_7) cannot be made to execute after (q_1, q_2) simply by reordering gates in E_{Q_2} because the two disconnected components in G_{I_2} can always execute in parallel regardless of gate order. To address this, we must add edges to G_{I_2} to connect the two components. For example, we can add edge (q_1, q_7) as shown in Figure 2(d), and denote the new interaction graph as G'_{I_2} .

A correct gate order necessitates that the first special gate (q_1, q_7) is executed before all gates in E'_{Q_2} , and the second special gate (q_0, q_7) execute after the other gates in E'_{Q_2} . To satisfy the first dependency constraint, we perform a breadth-first search (BFS) on G'_{I_2} , starting from q_1 and q_7 while ignoring the edge (q_0, q_7) . The order of visited edges from the BFS will establish the desired gate dependencies. In this case, the BFS result is $[(q_7, q_1), (q_0, q_4), (q_1, q_2), (q_0, q_3), (q_2, q_8), (q_0, q_6), (q_2, q_9), (q_2, q_5)]$. By prepending (q_1, q_7) to the BFS sequence, all gates will depend on it. To satisfy the second constraint, we run BFS on the same graph, starting from q_0 and q_7 , reverse the order of visited edges, and append (q_0, q_7) at the end. Concatenating these two sequences yields a circuit that meets both dependency constraints. While this method does not produce the smallest possible circuit, it is valid. For simplicity, we use a smaller sequence as an example. $C_2 = [g_5(q_4, q_7), g_6(q_1, q_2), g_7(q_4, q_0), g_8(q_8, q_2), g_9(q_0, q_3), g_{10}(q_2, q_9), g_{11}(q_0, q_6), g_{12}(q_2, q_5), g_{13}(q_2, q_1), g_{14}(q_1, q_7), g_{15}(q_0, q_7)]$.

Now, we can build a circuit C with an optimal SWAP count two by concatenating C_1 and C_2 , i.e., $C = C_1 \cdot C_2$. The gate dependency graph is illustrated in Figure 3, showing that gates from two interaction graphs must be executed in serial. We call C the *backbone* of the benchmark circuit and all gates between two special gates, including the later special gate, a *section* of the backbone circuit that enforce one SWAP gate.

We can introduce additional gates into the backbone without affecting the optimal SWAP gate count. For example, we can add (q_9, q_{10}) anywhere in C since it can be executed under any mapping. The key constraint is ensuring that additional gates do not introduce extra SWAP gates. For instance, (q_2, q_7) can only be inserted before g_4 since q_2 and q_7 are not neighbors after the first SWAP gate.

C. Challenges in Finding Optimal Solution for QUBIKOS

Unlike QUEKO [28], the optimal transpiled circuits for QUBIKOS benchmarks cannot be obtained using subgraph isomorphism algorithms, e.g., VF2 [30]. One may identify the special gates within the benchmark circuit by tracking the degrees of the interaction graph, and the special gates will partition the circuit into multiple sections that have an isomorphic subgraph of G . Nevertheless, running subgraph isomorphism algorithms on individual sections may not yield an optimal solution. For instance, applying VF2 to the first four gates only provides mapping information for a subset of qubits, and may not be optimal for the subsequent sections. As the optimal initial mapping for all qubits can only be

determined by examining the global structure of the circuit, this benchmark is effective for testing modern QLS tools.

D. Overview for Proof of Optimality

We provide an overview for proving the optimal SWAP count in QUBIKOS circuits, but have omitted the detail proofs due to the page limit. We list the four theorems that are used to prove the optimality:

Lemma 1. *Interaction graph $G_I(Q, E_Q)$ generated by subroutine 1 forms an interaction graph that is not isomorphic to any subgraph of $G_C(P, E_P)$.*

Lemma 2. $\forall g \in C_i$ generated by subroutine 2 has to execute after a prior special gate g_1 but before a subsequent special gate g_2 . (The prior special gate for the first section is Null.)

Lemma 3. $\forall 0 \leq i \leq n-1, \forall g \in C_i, \forall g' \in C_{i+1} : g \in \text{Prev}(g')$

Theorem 4. *The backbone circuit C needs at least n SWAP gates to execute.*

In Lemma 1, we show Section III-A always constructs a non-isomorphic interaction graph, which is trivial according to the construction. Then, we prove Lemma 2 by finding a path from g_1 to g and g to g_2 in the gate dependency graph. This is guaranteed by the BFS in Section III-B. Then, Lemma 3 states that any two gates belong to two consecutive sections of circuit C denoted as C_i and C_{i+1} must be executed in serial. This can be proved utilizing Lemma 2 by finding a path from the gate in C_i to a special gate, and a path from the special gate to the gate in C_{i+1} . Lastly, by Lemma 1 and 2, and 3, we can prove Theorem 4. Consider the backbone circuit depicted in Figure 3, we know there exists a two SWAP solution. Then, we verify that there does not exist a one SWAP solution using proof by contradiction. Assume a solution with one SWAP exists. Then, the solution will only have one SWAP gate inserted anywhere before the first special gate g_4 or after it. If the SWAP gate is inserted somewhere before g_4 , then the connectivity constraints imposed by the second interaction graph G'_{I_2} remain unresolved. On the other hand, adding a SWAP gate after g_4 can't resolve the connectivity constraints in G_{I_1} . Hence, such a solution cannot exist. For a general proof, we can just extend this argument to a circuit with n sections. By Theorem 4, we need at least n SWAP gates to execute C , and we know a solution with n SWAPs. Hence, we prove the optimality.

IV. EVALUATION

We verify the optimality of QUBIKOS in Section IV-A via OLSQ2 [10] with PySAT version 1.8.dev13 and Z3-solver version 4.13.3.0. In Section IV-B, we evaluate the optimality gap of existing QLS tool, LightSABRE from Qiskit [31] version 1.2.4, t|ket> [32] version 1.34.0, QMAP [33] version 2.7.0, and ML-QLS [27]. All the experiments were conducted on Intel Xeon CPU at 2.20GHz. For LightSABRE, we test it with 1000 trials.

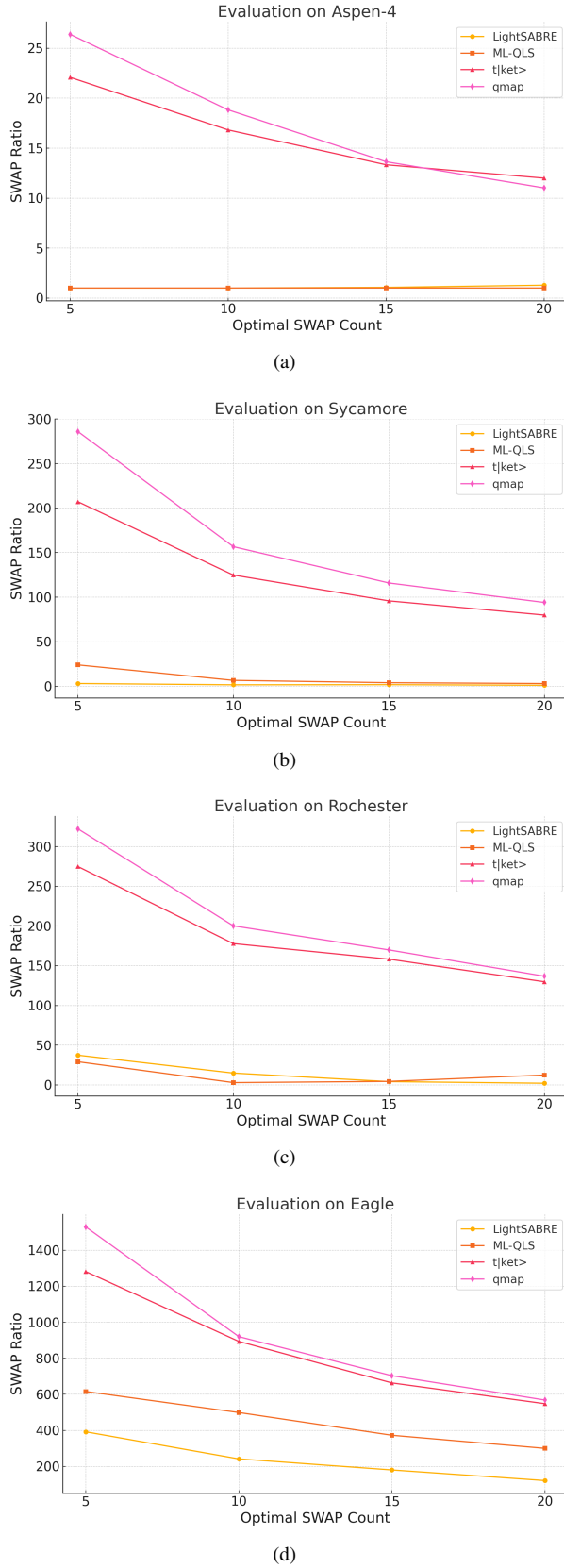


Figure 4. Results for the heuristic experiments. (a) shows the result for the Aspen-4 architecture. (b) reveals the result for the Sycamore architecture. (c) is the result for the Rochester architecture. (d) shows the result for the Eagle architecture.

A. Optimality Study

This experiment is to verify the optimal SWAP count for QUBIKOS circuits via the exact QLS tool OLSQ2 [10]. We generate QUBIKOS circuits for the Rigetti Aspen-4 with 16 qubits and 3x3 grid architectures. We create 400 circuits per architecture, with 100 for each SWAP count from 1 to 4. To guarantee a feasible runtime given OLSQ2's exponential complexity, we limit circuits to 30 two-qubit gates.

The optimality experiment confirms that all generated circuits have the desired SWAP counts. For each architecture, circuits designed with specific SWAP counts were verified by OLSQ2 to require the same SWAP counts, with no deviations observed. This consistency demonstrates that the benchmark circuits are optimally constructed.

B. QLS Tool Evaluation

This experiment is to measure the optimality gap of leading heuristic QLS tools. We produce QUBIKOS circuits for four architectures: Rigetti Aspen-4, Google Sycamore with 54 qubits, IBM Rochester with 53 qubits, and IBM Eagle with 127 qubits. For each architecture, we make 40 circuits with 10 for each SWAP count of 5, 10, 15, and 20. The two-qubit gate count is set to 300 for Aspen-4, 1500 for Sycamore and Rochester, and 3000 for Eagle. The reason for varying circuit size on different architectures is because a larger architecture requires more gates on average to construct a section of the backbone circuit as the interaction graph requiring more connections on average to be non-isomorphic.

The optimality gap is defined based on the SWAP ratio, which is calculated as the ratio of the average SWAP count to the optimal SWAP count: $\frac{\text{Average SWAP count}}{\text{Optimal SWAP count}}$. This ratio is always greater than or equal to 1, where a ratio of 1 indicates that the QLS tool finds the optimal solution.

Figure 4 presents the experiment results, showing how the optimality gap varies across different QLS tools. Among the four QLS tools, ML-QLS achieves optimal results on Aspen-4 and Rochester, and LightSABRE performs the best on the other architectures. ML-QLS performs similarly to LightSABRE, except on the Eagle architecture. This is different from the results in the ML-QLS paper [27] because we use a newer version of Qiskit and much higher trial number. So, LightSABRE has a better performance in our experiment but not in the ML-QLS experiment. In contrast, t|ket> and QMAP exhibit a significantly larger optimality gap with an average of 185x and 207x respectively on Aspen-4 and as high as 846x and 930x on Eagle.

As the number of physical qubits increases, the optimality gap also tends to grow. This is supported by the fact that on Aspen-4, ML-QLS finds the optimal solutions in all cases. But on Sycamore, Rochester, and Eagle, LightSABRE has an optimality gap of 1.95x, 12.17x, and 233.97x respectively. However, the number of physical qubits is not the only factor; the connectivity of the coupling graph also plays a significant role. For example, despite the Rochester architecture having a similar number of physical qubits to the Sycamore architecture, its optimality gap is six times larger. This is due to

Rochester’s heavy-hex structure, which has sparse connectivity and has fewer axes of symmetry compared to Sycamore’s dense grid connectivity. As a result, architectures with dense connectivity are more favorable for QLS tools, leading to smaller optimality gaps.

These findings highlight the utility of our examples for evaluating QLS tools, revealing that the optimality gap varies significantly depending on the tool. While LightSABRE and ML-QLS have smaller gaps, $t|ket\rangle$ and QMAP struggle to achieve near-optimal solutions. This emphasizes the need for more efficient QLS tools, particularly for larger architectures where the current tools demonstrate a large optimality gap. Additionally, the performance differences between the architectures suggest that the connectivity of the coupling graph influences how well QLS tools can minimize the SWAP count, with tools performing better on more connected architectures.

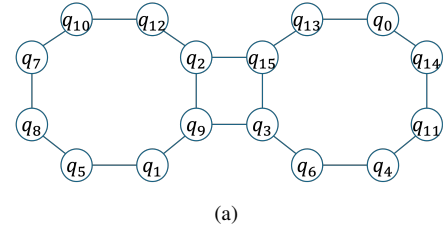
C. Case Study: LightSABRE

QUBIKOS not only provides a consistent evaluation of existing QLS tools but also offers insights for improving them. In this section, we perform a case study to analyze the results obtained from LightSABRE.

In one Aspen-4 experiment, SABRE identifies the optimal initial mapping but fails to find the correct routing, leading to a non-optimal solution. Figure 5 illustrates the partial circuit where SABRE deviates from the optimal routing. Instead of choosing to $\text{SWAP}(q_3, q_9)$, LightSABRE opts for $\text{SWAP}(q_2, q_9)$, resulting in additional SWAP gates and non-isomorphic mappings. To understand why LightSABRE chooses this suboptimal SWAP, we calculate the cost of the two SWAP gates. SABRE evaluates three types of costs: basic, lookahead, and decay. In this case, both SWAPs are used for the first time and connect q_2 and q_3 , so the basic and decay costs are the same. The difference lies in the lookahead cost. In LightSABRE, the lookahead is controlled by the extended set, where its size is the number of gates LightSABRE will consider beyond the execution layer. In Qiskit, the extended set size is 20 with a weight of 0.5. Thus, the lookahead cost for $\text{SWAP}(q_3, q_9)$ is 0.7, while for $\text{SWAP}(q_2, q_9)$, it is 0.65. This occurs because the qubit distance reduction for $\text{SWAP}(q_2, q_9)$ is greater when considering the first 20 gates equally, even though the two mappings cannot be executed under the same conditions, resulting in a non-optimal choice.

This issue could be mitigated by adding a decay factor to the lookahead cost, placing more weight on gates closer to the execution layer in D while reducing the weight for gates farther from execution. With an appropriate decay factor, SABRE could find the optimal routing.

The above analysis is specific to SABRE, but it can be done with any QLS tools to improve its performances. Moreover, this example reveals QUBIKOS circuit has a non-trivial qubit routing even when the initial mapping is optimal. This means QUBIKOS can also be utilized to evaluate standalone routers like [?] that require an initial mapping as input. Measuring the performance of these routers can be difficult since the quality of the initial mapping will directly impact its performance.



Gates in the extended set:	Circuit after optimal insertion:	Circuit after non-optimal insertion:
...	...	...
(q_2, q_3)	$\text{SWAP}(q_3, q_9)$	$\text{SWAP}(q_2, q_9)$
(q_3, q_2)	(q_2, q_3)	(q_2, q_3)
(q_2, q_{15})	(q_3, q_2)	(q_3, q_2)
(q_2, q_{12})	(q_2, q_{15})	$\text{SWAP}(q_9, q_{15})$
(q_9, q_2)	(q_2, q_{12})	(q_2, q_{15})
(q_9, q_2)	$\text{SWAP}(q_3, q_2)$	$\text{SWAP}(q_{15}, q_{12})$
(q_9, q_{15})	(q_9, q_2)	(q_2, q_{12})
(q_2, q_3)	(q_9, q_2)	$\text{SWAP}(q_{12}, q_9)$
(q_1, q_2)	(q_9, q_{15})	(q_9, q_2)
(q_2, q_9)	(q_2, q_3)	(q_9, q_2)
(q_2, q_3)	(q_1, q_2)	(q_9, q_{15})
(q_1, q_2)	(q_2, q_9)	(q_2, q_3)
(q_2, q_9)	(q_2, q_3)	(q_1, q_2)
(q_2, q_3)	(q_1, q_2)	(q_2, q_9)
(q_9, q_6)	(q_2, q_9)	(q_2, q_3)
(q_9, q_{15})	(q_2, q_3)	...
(q_3, q_{15})	...	...
(q_3, q_{12})	...	...
(q_{15}, q_{13})	...	...
(q_9, q_{15})	...	...
(q_2, q_3)	...	...
(q_9, q_2)	...	...
(q_1, q_2)	...	...

(b)

Figure 5. (a). Qubit mapping before LightSABRE generates non-optimal SWAP gate. (b). Portion of the circuit that LightSABRE inserts non-optimal SWAP. The left circuit is the benchmark circuit before insertion. It includes 20 gates due to the extended set size. The middle one is the circuit with optimal SWAP insertion. The right one is the non-optimal results from SABRE.

With QUBIKOS, we can assess the routers by providing the initial mapping of the generated circuit. This way, we can test the routers with the optimal initial mapping, and any non-optimal results from the routers directly relates to the design of the router itself rather than the initial mapping.

V. CONCLUSION

In this work, we introduce QUBIKOS, a benchmark with a known optimal SWAP count. We detail the method for generating QUBIKOS circuit and provide an overview for the proof. We verify its optimality experimentally and use it to evaluate the performance of four QLS tools. Our experiment results show that LightSABRE delivers the best performance but there is still an optimality gap that grows with the size of the architecture. Additionally, we illustrate how QUBIKOS can aid in improving QLS tools by analyzing a non-optimal case in LightSABRE, highlighting its potential to drive the development of more efficient QLS tools in the future.

VI. ACKNOWLEDGMENT

This research is supported in part by the National Science Foundation Award 2016245, 2313083, and the Harvard Quantum Initiative Postdoctoral Fellowship.

REFERENCES

- [1] M. Y. Siraichi *et al.*, “Qubit allocation.” New York, NY, USA: Association for Computing Machinery, Feb. 2018, pp. 113–125.
- [2] R. Wille, A. Lye, and R. Drechsler, “Optimal SWAP gate insertion for nearest neighbor quantum circuits,” in *2014 19th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2014, pp. 489–494.
- [3] D. Bhattacharjee, A. A. Saki, M. Alam, A. Chattopadhyay, and S. Ghosh, “MUQUT: Multi-constraint quantum circuit mapping on NISQ computers,” in *2019 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 2019, pp. 1–7.
- [4] R. Wille *et al.*, “Mapping quantum circuits to IBM QX architectures using the minimal number of SWAP and H operations,” in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.
- [5] B. Tan and J. Cong, “Optimal layout synthesis for quantum computing,” in *Proceedings of the 39th International Conference on Computer-Aided Design*. ACM, Nov. 2020, pp. 1–9.
- [6] —, “Optimal qubit mapping with simultaneous gate absorption,” in *2021 IEEE/ACM International Conference on Computer Aided Design*. IEEE, 2021, pp. 1–8.
- [7] C. Zhang *et al.*, “Time-optimal Qubit mapping.” New York, NY, USA: Association for Computing Machinery, Apr. 2021, pp. 360–374.
- [8] A. Molavi *et al.*, “Qubit mapping and routing via MaxSAT,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2022, pp. 1078–1091.
- [9] G. Nannicini *et al.*, “Optimal qubit assignment and routing via integer programming,” *ACM Transactions on Quantum Computing*, vol. 4, no. 1, pp. 1–31, 2022.
- [10] W.-H. Lin *et al.*, “Scalable optimal layout synthesis for NISQ quantum processors,” in *2023 60th ACM/IEEE Design Automation Conference*. IEEE, 2023, pp. 1–6.
- [11] A. Ho and D. Bacon, “Announcing Cirq: an open source framework for NISQ algorithms,” *Google AI Blog*, vol. 18, 2018.
- [12] A. Zulehner *et al.*, “An efficient methodology for mapping quantum circuits to the IBM QX architectures,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 7, pp. 1226–1236, 2018.
- [13] IBM. (2018) Qiskit. [Online]. Available: <https://qiskit.org/>
- [14] A. Zulehner *et al.*, “An Efficient Methodology for Mapping Quantum Circuits to the IBM QX Architectures,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 7, pp. 1226–1236, Jul. 2019.
- [15] M. Y. Siraichi *et al.*, “Qubit allocation as a combination of subgraph isomorphism and token swapping,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, pp. 120:1–120:29, Oct. 2019.
- [16] G. Li *et al.*, “Tackling the qubit mapping problem for NISQ-era quantum devices.” New York, NY, USA: Association for Computing Machinery, Apr. 2019, pp. 1001–1014.
- [17] P. Murali *et al.*, “Formal constraint-based compilation for noisy intermediate-scale quantum systems,” *Microprocessors & Microsystems*, vol. 66, no. C, pp. 102–112, Apr. 2019.
- [18] S. Sivarajah *et al.*, “[t]ket): a retargetable compiler for NISQ devices,” *Quantum Science and Technology*, vol. 6, no. 1, p. 014003, Nov. 2020.
- [19] A. Kole *et al.*, “Improved Mapping of Quantum Circuits to IBM QX Architectures,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2375–2383, Oct. 2020.
- [20] J. Liu *et al.*, “Not all SWAPs have the same cost: a case for optimization-aware qubit routing,” in *2022 IEEE International Symposium on High-Performance Computer Architecture*, Apr. 2022, pp. 709–725, ISSN: 2378-203X.
- [21] T.-A. Wu *et al.*, “A robust quantum layout synthesis algorithm with a qubit mapping checker,” in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, ser. ICCAD ’22. New York, NY, USA: Association for Computing Machinery, 2022.
- [22] H. Fan *et al.*, “Optimizing quantum circuit placement via machine learning,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 19–24.
- [23] C.-Y. Huang *et al.*, “Reinforcement learning and dear framework for solving the qubit mapping problem,” in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 2022, pp. 1–9.
- [24] S. Park *et al.*, “A fast and scalable qubit-mapping method for noisy intermediate-scale quantum computers,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, New York, NY, USA, 2022, p. 13–18.
- [25] C.-Y. Huang and W.-K. Mak, “CTQR: Control and timing-aware qubit routing,” in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 140–145.
- [26] —, “Efficient qubit routing using a dynamically-extract-and-route framework,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [27] W.-H. Lin and J. Cong, “ML-QLS: Multilevel quantum layout synthesis,” *arXiv preprint arXiv:2405.18371*, 2024.
- [28] B. Tan and J. Cong, “Optimality study of existing quantum computing layout synthesis tools,” *IEEE Transactions on Computers*, vol. 70, no. 9, pp. 1363–1373, 2020.
- [29] S. Li, X. Zhou, and Y. Feng, “On constructing benchmark quantum circuits with known near-optimal transformation cost,” 2023. [Online]. Available: <https://arxiv.org/abs/2301.08932>
- [30] L. Cordella, P. Foggia, C. Sansone, and M. Vento, “A (sub)graph isomorphism algorithm for matching large graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 26, no. 10, pp. 1367–1372, 2004.
- [31] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, “LightSABRE: A lightweight and enhanced SABRE algorithm,” *arXiv preprint arXiv:2409.08368*, 2024.
- [32] A. Cowtan, S. Dilkes, R. Duncan, A. Krajenbrink, W. Simmons, and S. Sivarajah, “On the qubit routing problem,” *arXiv preprint arXiv:1902.08091*, 2019.
- [33] R. Wille and L. Burgholzer, “MQT QMAP: Efficient quantum circuit mapping,” in *Proceedings of the 2023 International Symposium on Physical Design*, ser. ISPD ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 198–204. [Online]. Available: <https://doi.org/10.1145/3569052.3578928>